# intro

October 12, 2026

## 1 intro to statsmodels

### 1.0.1 Key features

- Regression Models: Supports different types of regression, such as ordinary least squares (OLS) and generalized least squares (GLS)
- Time Series Analysis: Provides models like ARIMA, VAR, and exponential smoothing for forecasting time-based data
- Statistical Tests: Includes t-tests, chi-square tests, and normality tests to check data patterns and relationships
- Nonparametric Methods: Helps analyze data without assuming a specific distribution using methods like kernel density estimation (KDE)
- Robust Models: Offers models that handle outliers and data that do not follow standard assumptions

### 1.0.2 Linear Regression

- Statsmodels offers ordinary least squares (OLS) regression, a common method for studying relationships between variables.
- It helps you understand how an independent variable (X) affects a dependent variable (y).
- OLS finds the best-fitting line by minimizing the difference between the actual and predicted values.

[1]:
```
import statsmodels.api as sm
import numpy as np
import pandas as pd

# Generate sample data
np.random.seed(42)
X = np.random.rand(100, 1)
y = 3 * X.squeeze() + np.random.randn(100) * 0.5

# Add a constant for the intercept
X = sm.add_constant(X)

# Fit the OLS model
model = sm.OLS(y, X).fit()

# Print model summary
```

```
print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.769
Model:                            OLS   Adj. R-squared:                  0.767
Method:                 Least Squares   F-statistic:                     326.7
Date:                Mon, 12 Oct 2026   Prob (F-statistic):           5.66e-33
Time:                        10:07:03   Log-Likelihood:                -61.832
No. Observations:                 100   AIC:                             127.7
Df Residuals:                      98   BIC:                             132.9
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          0.1075      0.085      1.263      0.210      -0.061       0.277
x1             2.7701      0.153     18.076      0.000       2.466       3.074
==============================================================================
Omnibus:                        0.900   Durbin-Watson:                   2.285
Prob(Omnibus):                  0.638   Jarque-Bera (JB):                0.808
Skew:                           0.217   Prob(JB):                        0.668
Kurtosis:                       2.929   Cond. No.                         4.18
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
```

### 1.0.3 Time Series Analysis

- Time series analysis helps identify patterns, trends, and seasonal effects in time-dependent data.
- Statsmodels offers models like ARIMA, VAR, and exponential smoothing for time series forecasting.

[2]:
```
from statsmodels.tsa.arima.model import ARIMA

# Generate sample time series data
ts_data = np.cumsum(np.random.randn(100))

# Fit ARIMA model
model = ARIMA(ts_data, order=(1, 1, 1)).fit()

# Print model summary
print(model.summary())
```

```
                               SARIMAX Results
==============================================================================
```

```
Dep. Variable:                      y   No. Observations:                  100
Model:                 ARIMA(1, 1, 1)   Log Likelihood                -149.579
Date:                Mon, 12 Oct 2026   AIC                            305.158
Time:                        10:07:46   BIC                            312.944
Sample:                             0   HQIC                           308.308
                                  - 100
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1          0.3025      1.843      0.164      0.870      -3.310       3.915
ma.L1         -0.3554      1.794     -0.198      0.843      -3.871       3.160
sigma2         1.2019      0.139      8.641      0.000       0.929       1.474
===================================================================================
===
Ljung-Box (L1) (Q):                   0.00   Jarque-Bera (JB):
6.12
Prob(Q):                              0.97   Prob(JB):
0.05
Heteroskedasticity (H):               1.06   Skew:
0.34
Prob(H) (two-sided):                  0.86   Kurtosis:
4.01
===================================================================================
===

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-
step).
```

### 1.0.4 Statistical Tests

- Statistical tests help make decisions or inferences about a population based on sample data.
- Statsmodels offers several statistical tests, such as t-tests, ANOVA, normality tests, and chi-square tests.

[3]:
```python
from statsmodels.stats.weightstats import ttest_ind
import numpy as np

# Generating sample data
data1 = np.random.randn(100)
data2 = np.random.randn(100)

# Performing T-test
t_stat, p_value, df = ttest_ind(data1, data2)
print(f"T-statistic: {t_stat}, P-value: {p_value}")
```

```
T-statistic: 1.1668948366355638, P-value: 0.24465628315141202
```

[ ]: